# EL-2003 – Computer Organization & Assembly Language Lab
# Lab # 7

**Objectives:**

1. Explain how the stack works (last in, first out, growing downwards) and predict SP after each push, pop, call and ret.
2. Use push and pop to save and restore registers, reverse values and swap registers.
3. Write subroutines with call and ret and place them correctly in the program.
4. Pass parameters to a subroutine through registers and through the stack, reading them with BP and removing them with ret n.
5. Preserve the caller's registers and return answers in AX.
6. Use the stack as a data structure to reverse a string, and follow nested calls.
7. Verify results in AFD by watching SP, the Stack list and the memory dump.

**Note: Carefully read the following instructions (*Each instruction contains a weightage*)**

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

---

**Question 1:**

At the very start of the program, store the value of SP in a word variable sp_start (in a .com program it is FFFEh). Load AX = 1111h, BX = 2222h and CX = 3333h, then push AX, BX and CX in that order. Store SP again in sp_after (expected FFF8h, because three pushes use 6 bytes). Next, pop three values into DX, SI and DI, in that order, and store them in an array rev of three words (DX first). The expected values are 3333h, 2222h and 1111h. Store SP once more in sp_end (expected FFFEh). Finally, swap the contents of AX and BX using only push and pop, with no mov between registers and no xchg, and store AX and BX in swapped (expected 2222h and 1111h).

**Question 2:**

Write a subroutine square that receives a number (at most 255) in AX and returns its square in AX. It must not change any other register, so it should save and restore every register it uses with push and pop. Declare nums: db 3, 5, 7, 10, 12, 15 and squares: dw 0, 0, 0, 0, 0, 0. Store SP in sp_in before the loop. Load DX with BEEFh, then loop over the six numbers: load each into AL, clear AH, call square, and store the result in the matching word of squares. After the loop, store DX in dx_after (expected BEEFh, which proves square did not damage it) and store SP in sp_out. The expected squares are 9, 25, 49, 100, 144 and 225, and sp_out must equal sp_in. Place the subroutine after the exit code.

**Question 3:**

Write a subroutine maxof3 that receives three word values through the stack, where the caller pushes a first, then b, then c. The subroutine returns the largest value (unsigned) in AX and removes its own parameters from the stack with ret 6. Use BP to read the parameters. Declare tests: dw 45, 90, 67, 12, 7, 5, 30, 30, 31 (three sets of a, b and c) and maxres: dw 0, 0, 0. In a loop, push each set from registers (a number cannot be pushed directly on an 8086), call maxof3, and store the answer in maxres. Store SP in sp_in before the loop and in sp_out after it. The expected results are 005Ah, 000Ch and 001Fh, and sp_out must equal sp_in.

**Question 4:**

A library charges 20 rupees for each day a book is late, but a fine can never be more than 500 rupees. The late days of five members are stored as lateDays: dw 3, 0, 12, 30, 7. The librarian wants one subroutine that works out a single member's fine, and a program that uses it for all five members. The days late and the daily rate must be passed to the subroutine through the stack, not through registers. Decide how the answer comes back, who removes the parameters from the stack, and which registers the subroutine must protect. Store each member's fine in an array fines, store the total of all fines in totalfine, and store the number of members whose fine reached the cap in capcount. The stack must be balanced when the program ends.

**Question 5:**

A parcel office stores tracking codes that are always exactly five letters long. Four codes are saved back to back as codes: db "RADAR", "HELLO", "LEVEL", "STACK". The office wants to know which codes read the same backwards. For each code, use the stack to build the reversed code in a second block of memory called reversed (20 bytes in total), and set a flag in a four-byte array ispal to 1 if the code is the same backwards, or 0 if it is not. Write the reversing work as a subroutine of your own design, protecting any registers it uses, and make sure the stack is balanced when the program ends.